

HW_03

Henry Traynor

Due: 5:00pm, 8 September

Create a self-contained R script or Rmarkdown/Quarto file to do the following:

1. Fit an occupancy model in 'unmarked' to the Ruffed Grouse data ("grouse_data.csv"). Use all three covariates, with elevation and forest cover affecting occupancy and day of the year affecting detection probability.

- Response: grouse1, grouse2, grouse3
- Site covs: Elevation (elev), forest cover (fc)
- Obs cov: Day of the year (doy.1, doy.2, doy.3)

```
setwd("C:/Users/henry/Desktop/WILD8390/Homework/HW03")
grouse_data <- read.csv("grouse_data.csv")
```

```
library(unmarked)
umf <- unmarkedFrameOccu(y = grouse_data[,c("grouse1", "grouse2", "grouse3")],
  siteCovs = grouse_data[,c("elev", "fc")],
  obsCovs = list(doy=grouse_data[,c("doy.1", "doy.2", "doy.3")])
)
summary(umf)
```

```
## unmarkedFrame Object
##
## 1267 sites
## Maximum number of observations per site: 3
## Mean number of observations per site: 3
## Sites with at least one detection: 26
##
## Tabulation of y observations:
##      0      1
## 3772    29
##
## Site-level covariates:
##      elev      fc
## Min.   : 312.6   Min.   :13.66
## 1st Qu.: 501.8   1st Qu.:54.42
## Median : 602.9   Median :68.79
## Mean    : 647.3   Mean    :64.60
## 3rd Qu.: 755.2   3rd Qu.:76.96
## Max.    :1347.9   Max.    :85.00
##
```

```

## Observation-level covariates:
##      doy
## Min.   : 52.0
## 1st Qu.: 83.0
## Median :100.0
## Mean   :100.3
## 3rd Qu.:117.0
## Max.   :146.0

# standarize cont. variables
mean.elev <- mean(grouse_data[, "elev"])
mean.forest <- mean(grouse_data[, "fc"])
sd.elev <- sd(grouse_data[, "elev"])
sd.forest <- sd(grouse_data[, "fc"])
elev_s <- (grouse_data[, "elev"] - mean.elev) / sd.elev
forest_s <- (grouse_data[, "fc"] - mean.forest) / sd.forest
siteCovs(umf)$elev_s <- elev_s
siteCovs(umf)$forest_s <- forest_s

# fit the model
fm <- occu(~elev_s+doy ~elev_s+forest_s, umf) # Standardized covariates
fm

##
## Call:
## occu(formula = ~elev_s + doy ~ elev_s + forest_s, data = umf)
##
## Occupancy (logit-scale):
##      Estimate      SE      z P(>|z|)
## (Intercept) -2.898 1.350 -2.147 0.0318
## elev_s      0.629 0.766  0.821 0.4118
## forest_s    0.773 0.377  2.051 0.0403
##
## Detection (logit-scale):
##      Estimate      SE      z P(>|z|)
## (Intercept) -3.70888 1.62050 -2.289 0.0221
## elev_s      0.44775 0.76233  0.587 0.5570
## doy         0.00932 0.00918  1.015 0.3100
##
## AIC: 300.7825
## Number of sites: 1267

# prediction
pred.data <- data.frame(elev_s=seq(from=-3, to=3, length=100),
                        forest_s=0, doy=100) # Standardized
pred.data$elev <- pred.data$elev_s*sd.elev+mean.elev # Orig scale
pred.data$fc <- pred.data$forest_s*sd.forest+mean.forest

psi.pred <- predict(fm, newdata=pred.data,
                    type='state', append=TRUE)
p.pred <- predict(fm, newdata=pred.data,
                  type='det', append=TRUE)

print(head(psi.pred, n=4), digits=2)

```

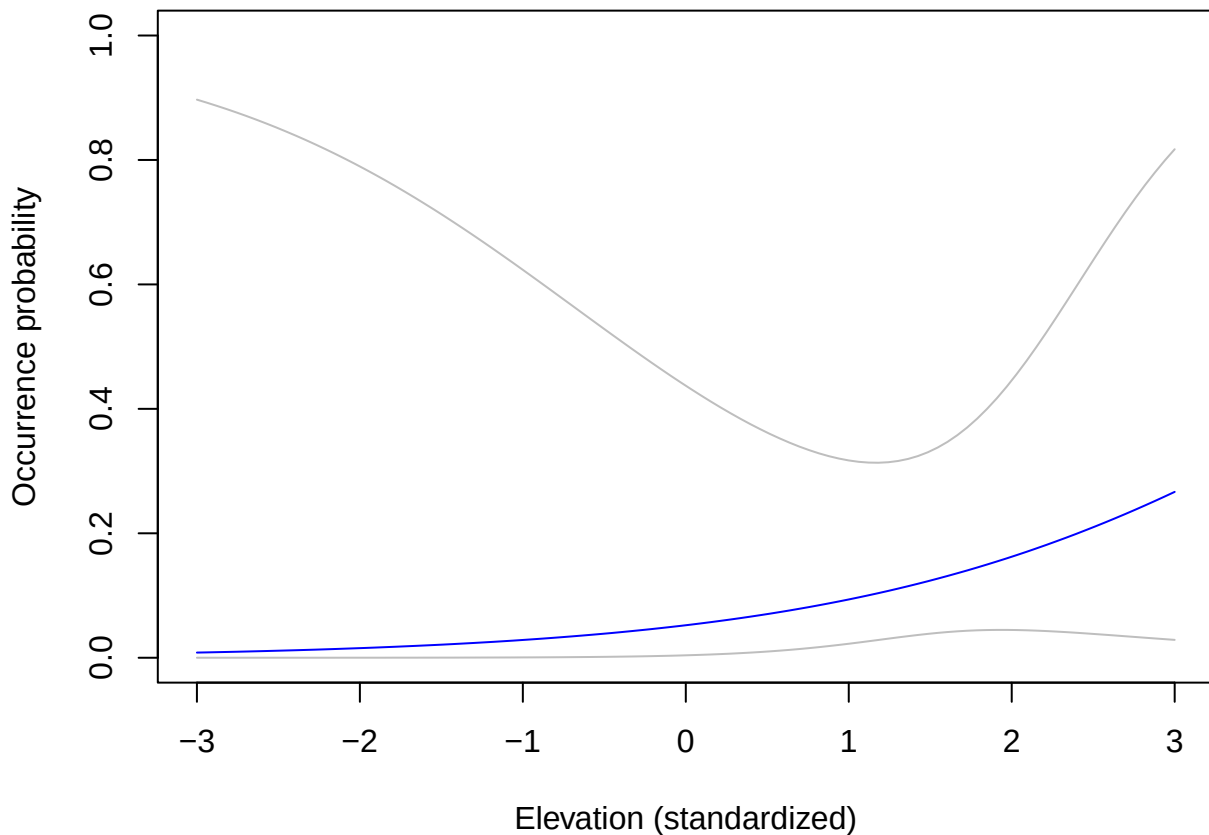
```
## Predicted SE lower upper elev_s forest_s doy elev fc
## 1 0.0083 0.029 8.0e-06 0.90 -3.0 0 100 66 65
## 2 0.0086 0.030 9.1e-06 0.89 -2.9 0 100 77 65
## 3 0.0089 0.031 1.0e-05 0.89 -2.9 0 100 89 65
## 4 0.0093 0.031 1.2e-05 0.88 -2.8 0 100 101 65
```

```
print(head(p.pred, n=4), digits=2)
```

```
## Predicted SE lower upper elev_s forest_s doy elev fc
## 1 0.016 0.056 1.5e-05 0.95 -3.0 0 100 66 65
## 2 0.016 0.057 1.7e-05 0.94 -2.9 0 100 77 65
## 3 0.017 0.057 1.9e-05 0.94 -2.9 0 100 89 65
## 4 0.017 0.058 2.2e-05 0.93 -2.8 0 100 101 65
```

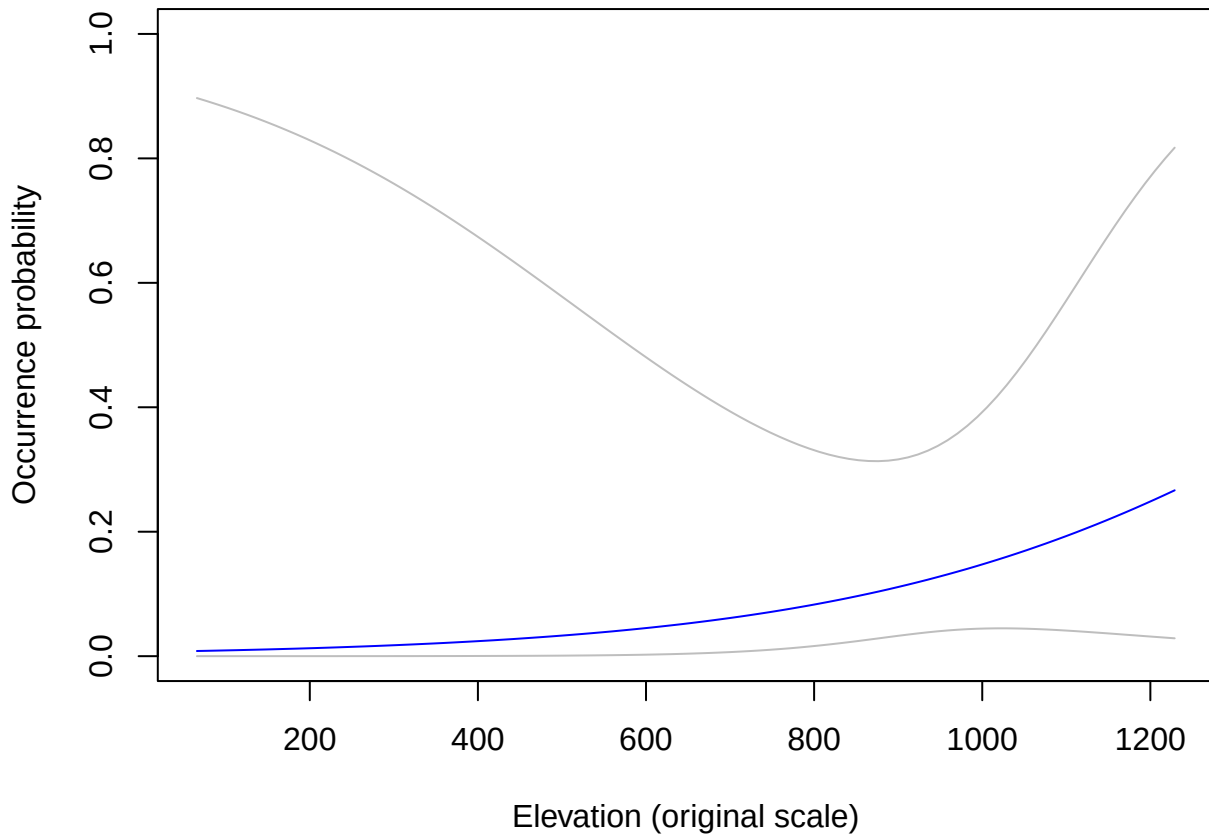
2. Make one prediction graph for occupancy and one for detection.

```
# these plots might not generate here but will if you knit it. PDF is also available at github link at
par(mai=c(0.9,0.9,0.1,0.1))
plot(Predicted ~ elev_s, psi.pred, type="l", ylab="Occurrence probability",
     col="blue", xlab="Elevation (standardized)", ylim=0:1)
lines(lower ~ elev_s, psi.pred, col="grey"); lines(upper ~ elev_s, psi.pred, col="grey")
```



```
par(mai=c(0.9,0.9,0.1,0.1))
plot(Predicted ~ elev, psi.pred, type="l", ylab="Occurrence probability",
```

```
col="blue", xlab="Elevation (original scale)", ylim=0:1)
lines(lower ~ elev, psi.pred, col="grey"); lines(upper ~ elev, psi.pred, col="grey")
```



3. Create the same graphs as above, but using Bayesian methods.

```
model_string <- "
model {

  # Priors for occupancy coefficients
  beta0 ~ dnorm(0, 0.5) # 0.5 is 1/variance
  beta1 ~ dnorm(0, 0.5)
  beta2 ~ dnorm(0, 0.5)

  # Priors for detection coefficients
  alpha0 ~ dnorm(0, 0.5)
  alpha1 ~ dnorm(0, 0.5)

  for(i in 1:nSites) {
    logit(psi[i]) <- beta0 + beta1*elev[i] + beta2*forest[i]
    z[i] ~ dbern(psi[i]) # Latent presence/absence
    for(j in 1:nOccasions) {
      logit(p[i,j]) <- alpha0 + alpha1*doy[i,j]
      y[i,j] ~ dbern(z[i]*p[i,j]) # Data
    }
  }
}
```

```

sitesOccupied <- sum(z)

}"

nSites <- nrow(grouse_data)
nVisits <- 3

jags.data <- list(y=grouse_data[,c("grouse1", "grouse2", "grouse3")],
                 elev=elev_s,
                 forest=forest_s,
                 doy=grouse_data[,c("doy.1", "doy.2", "doy.3")],
                 nSites=nSites,
                 nOccasions=nVisits)

jags.inits <- function() {
  list(beta0=rnorm(1), alpha0=rnorm(1), z=rep(1, nSites))
}

jags.pars <- c("beta0", "beta1", "beta2",
              "alpha0", "alpha1",
              "sitesOccupied")

library(jagsUI)
library(coda)

```

```

##
## Attaching package: 'coda'

## The following object is masked from 'package:jagsUI':
##
##      traceplot

```

```

jags.post.samples <- jags.basic(data=jags.data, inits=jags.inits,
                                parameters.to.save=jags.pars,
                                model.file="occupancy-model-covs.jag",
                                n.chains=3, n.adapt=100, n.burnin=0,
                                n.iter=2000, parallel=TRUE)

```

```

##
## Processing function input.....
##
## Converted data.frame y to matrix
##
## Converted data.frame doy to matrix
##
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.308 minutes.

```

```

### -----
### RUN THESE LINES IN YOUR CONSOLE
### -----
png("jags_plot.png", width = 500, height = 800)
plot(jags.post.samples)

psi.coef.post <- as.matrix(jags.post.samples[,c("beta0","beta1","beta2")])
head(psi.coef.post, n=4)

##           beta0      beta1      beta2
## [1,] -2.723105  0.8534720  0.2282817
## [2,] -2.825366  1.0027030  0.1899948
## [3,] -2.838383  0.9975852  0.1878531
## [4,] -2.945100  1.2618714  0.0540294

n.iter <- nrow(psi.coef.post)
psi.post.pred <- matrix(NA, nrow=n.iter, ncol=nrow(pred.data))

for(i in 1:n.iter) {
  psi.post.pred[i,] <- plogis(psi.coef.post[i,"beta0"] +
                             psi.coef.post[i,"beta1"]*pred.data$elev_s)
}

plot(pred.data$elev, psi.post.pred[1,], type="l", xlab="Elevation",
      ylab="Occurrence probability", ylim=c(0, 1), col=gray(0.8))

for(i in 1:n.iter) {
  lines(pred.data$elev, psi.post.pred[i,], col=gray(0.8))
}

pred.post.mean <- colMeans(psi.post.pred)
pred.post.lower <- apply(psi.post.pred, 2, quantile, prob=0.025)
pred.post.upper <- apply(psi.post.pred, 2, quantile, prob=0.975)
lines(pred.data$elev, pred.post.mean, col="blue")
lines(pred.data$elev, pred.post.lower, col="blue", lty=2)
lines(pred.data$elev, pred.post.upper, col="blue", lty=2)

```